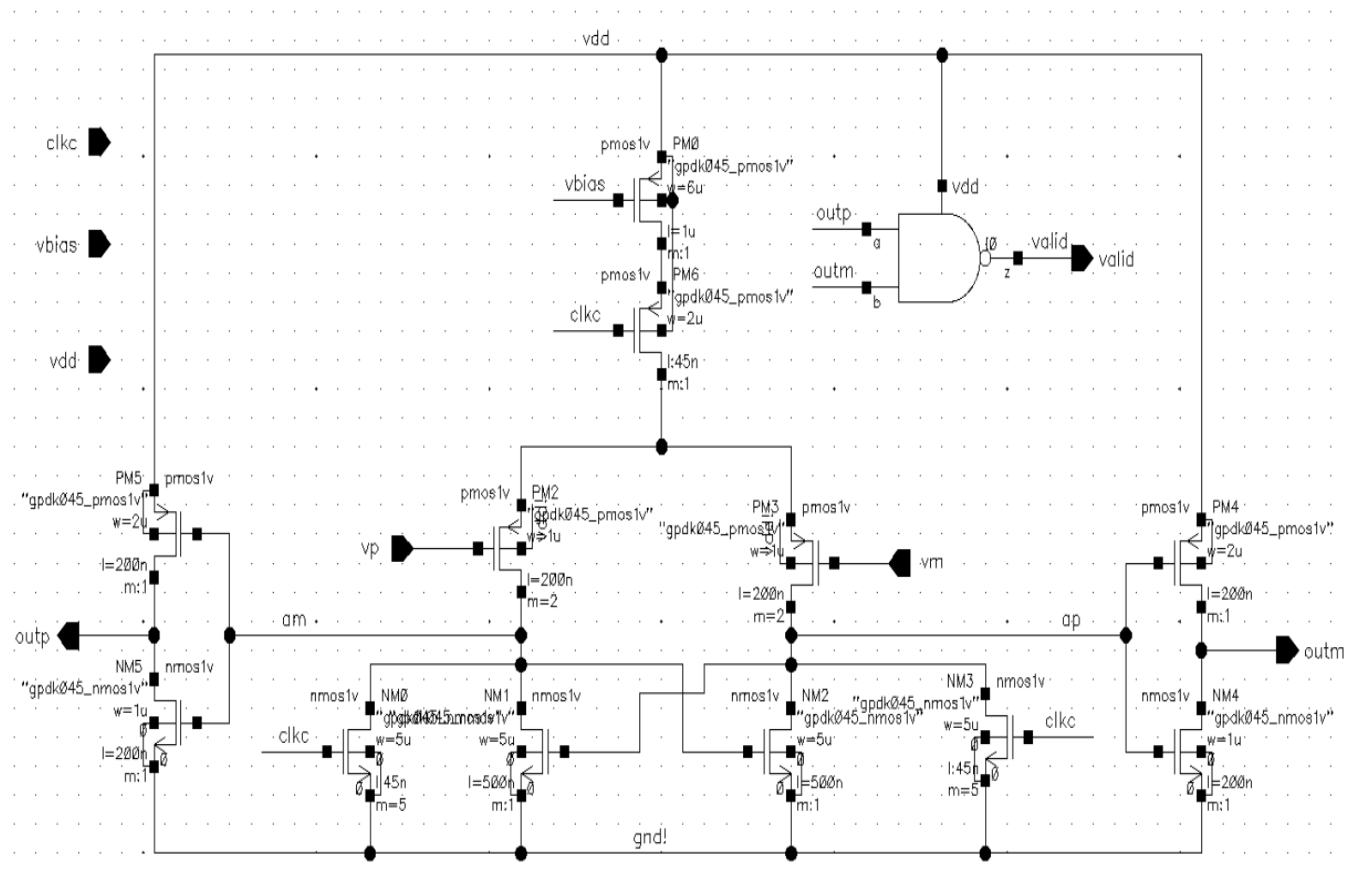


# 10MS/s 8-bit SAR ADC

Kareem Elrakabawy

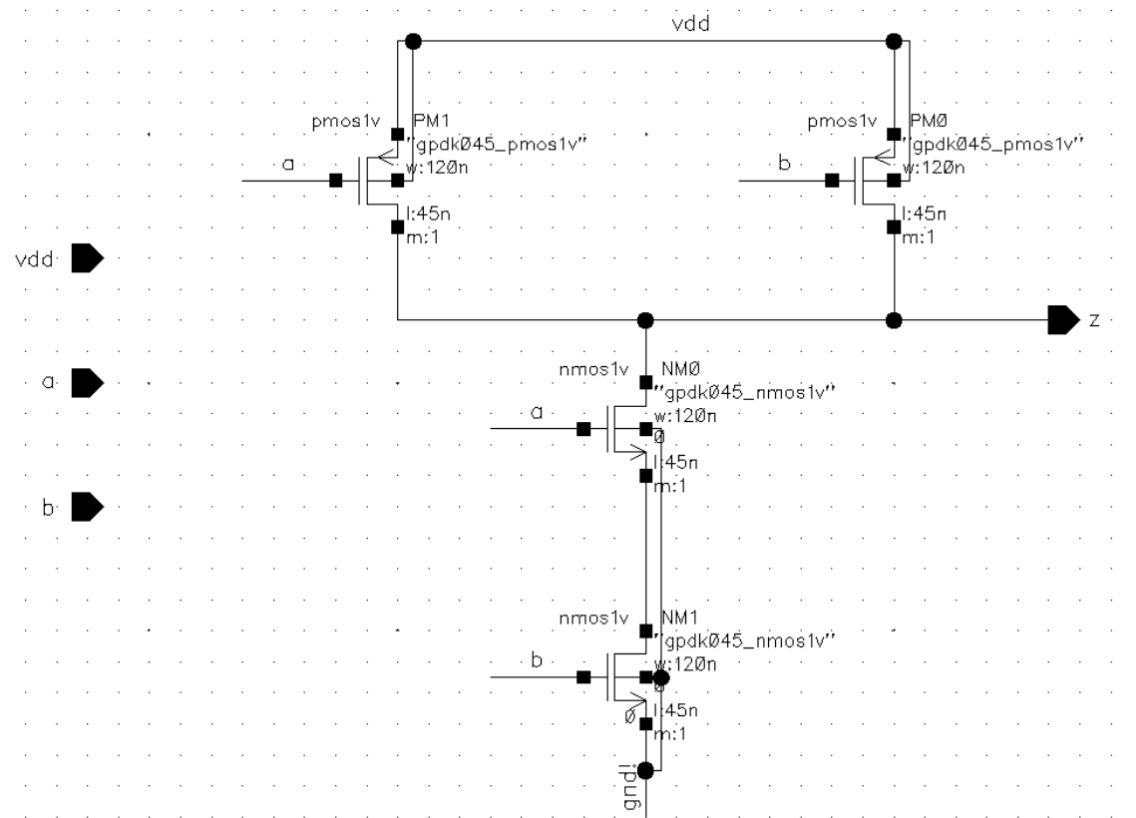
## Top Schematic

# Comparator (1/3)



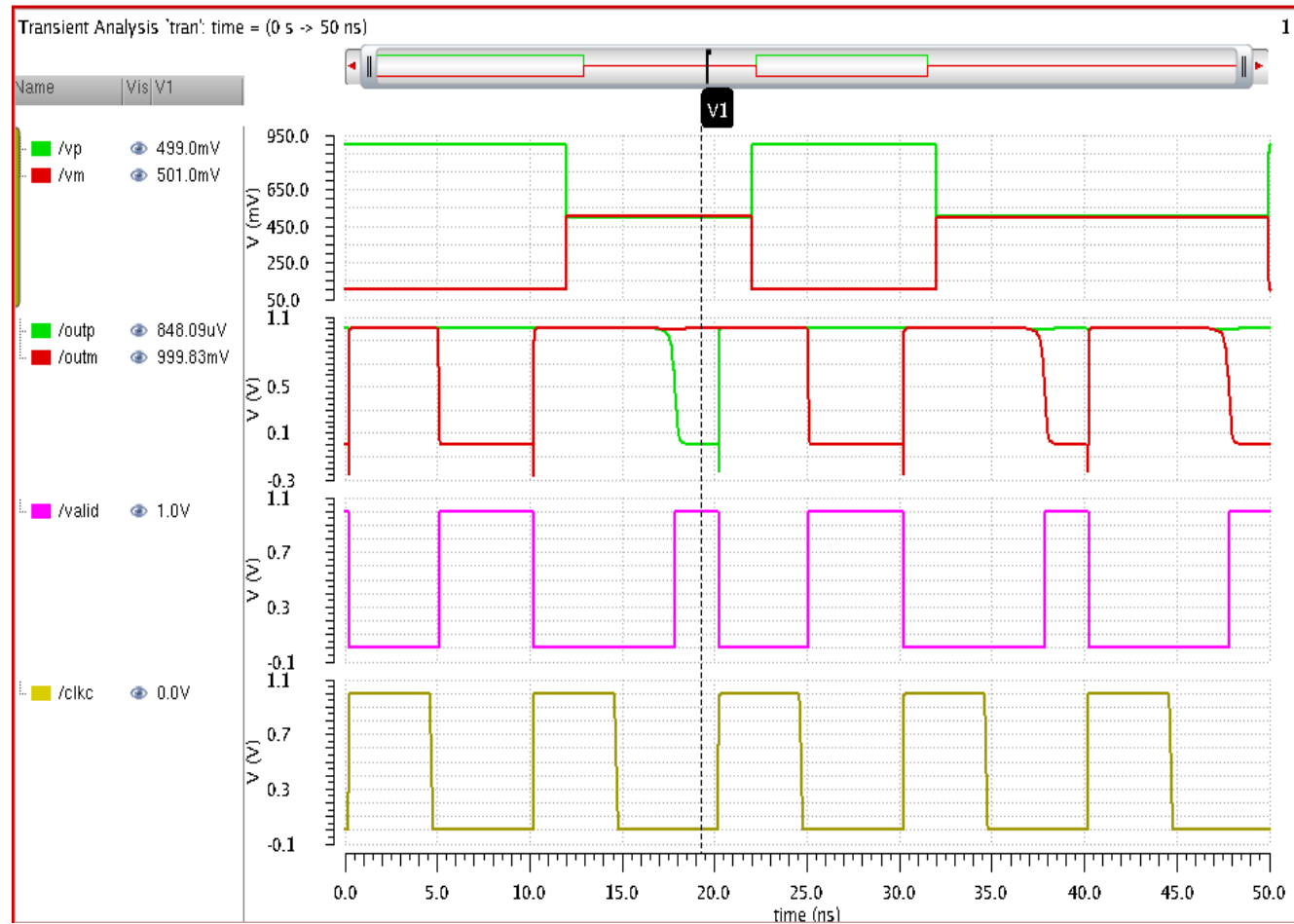
Dynamic Comparator Schematic

# Comparator (2/3)



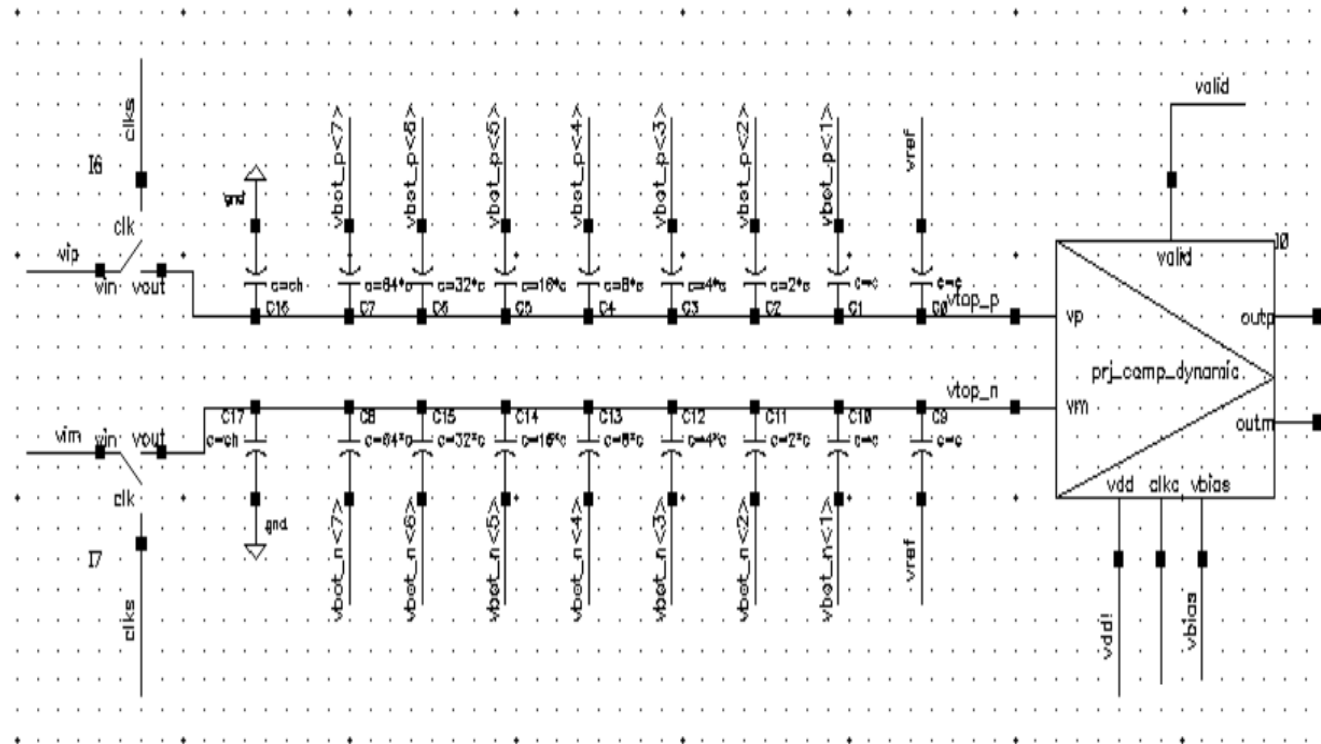
2x1 NAND

# Comparator (3/3)



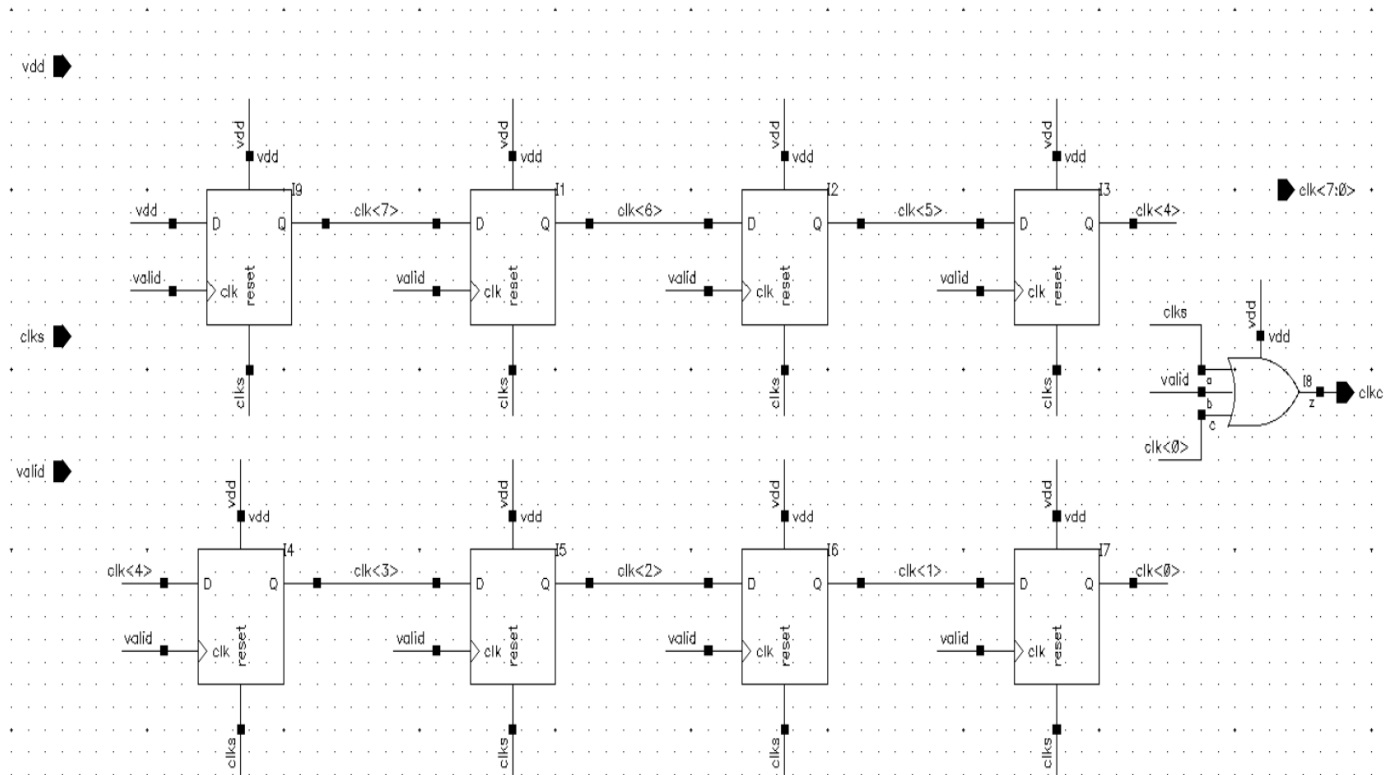
Overdrive Test Waveforms

# Capacitive DAC



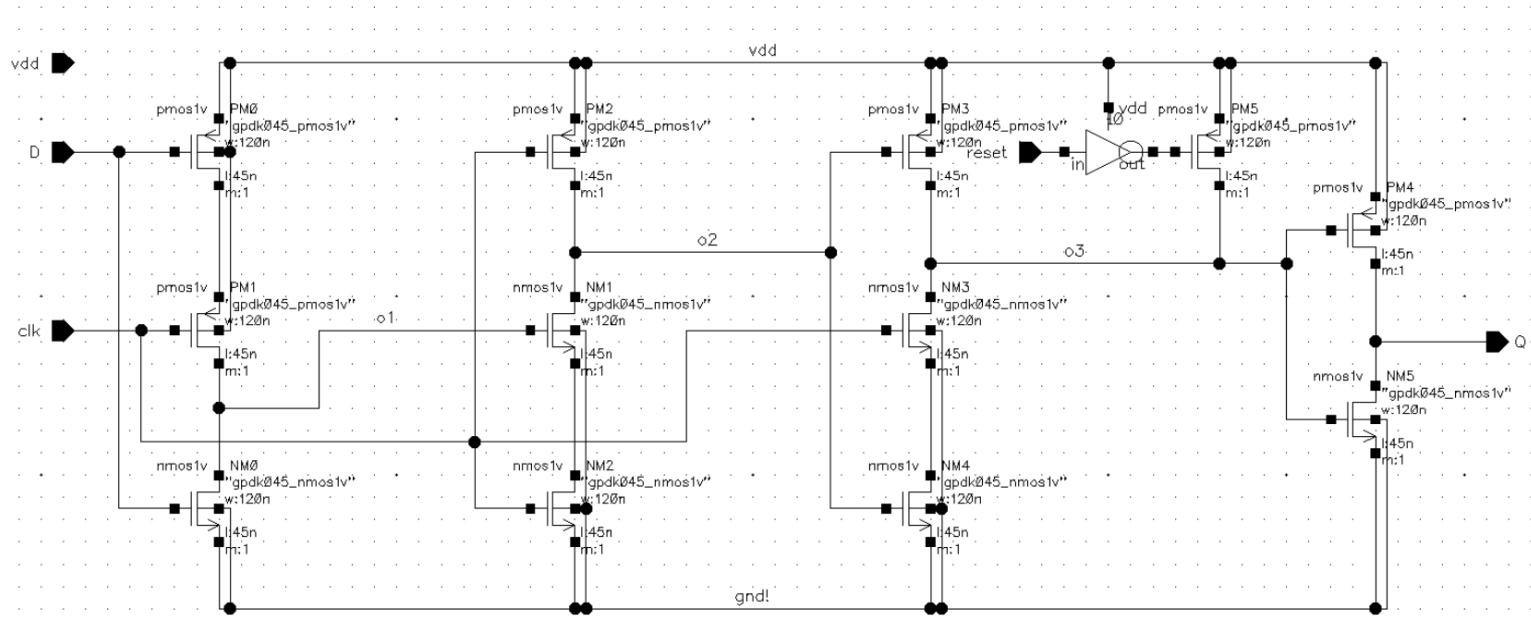
Charge-redistribution Network

# Asynchronous Clock Generator (1/5)



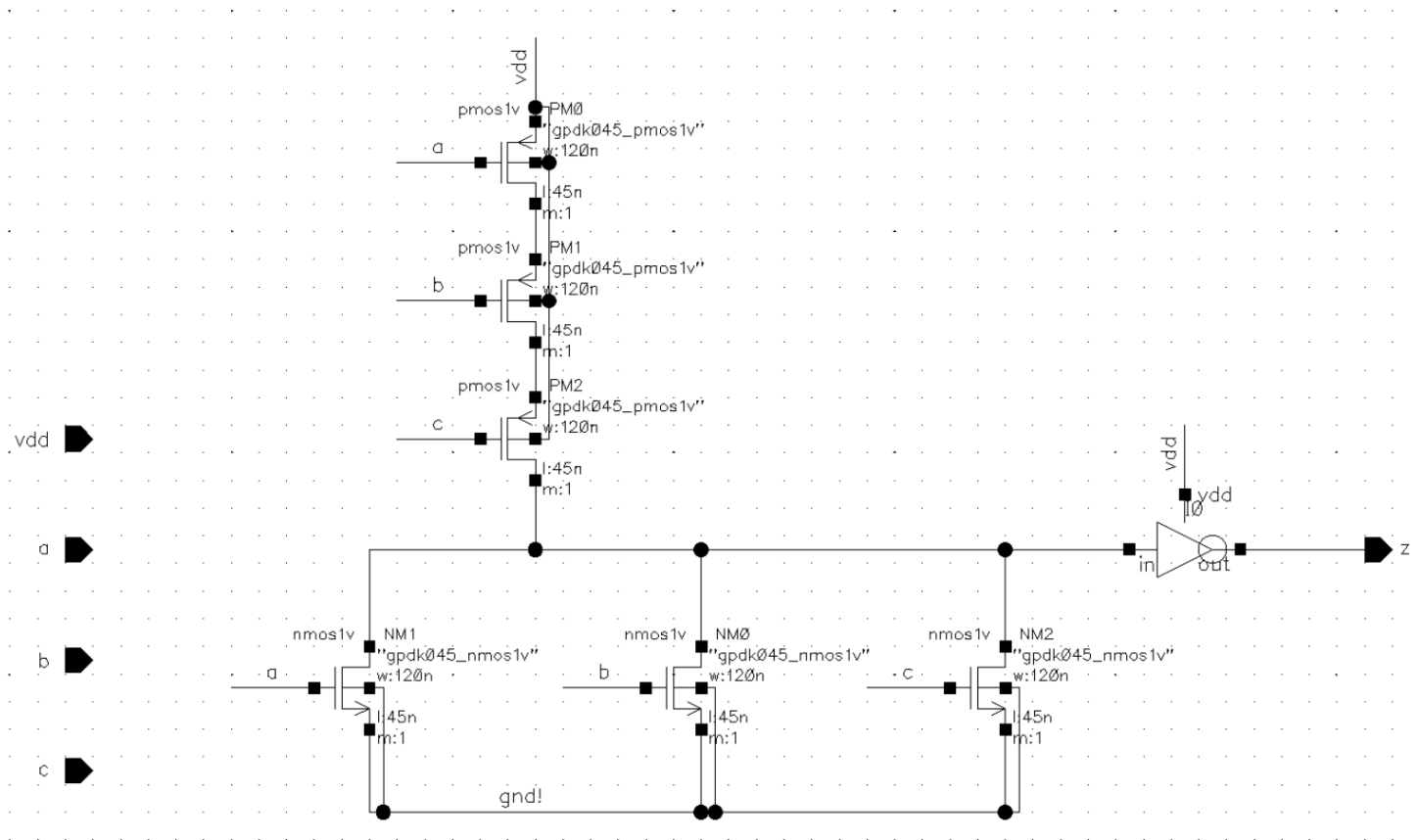
**Asynchronous Clock Generator Schematic**

# Asynchronous Clock Generator (2/5)



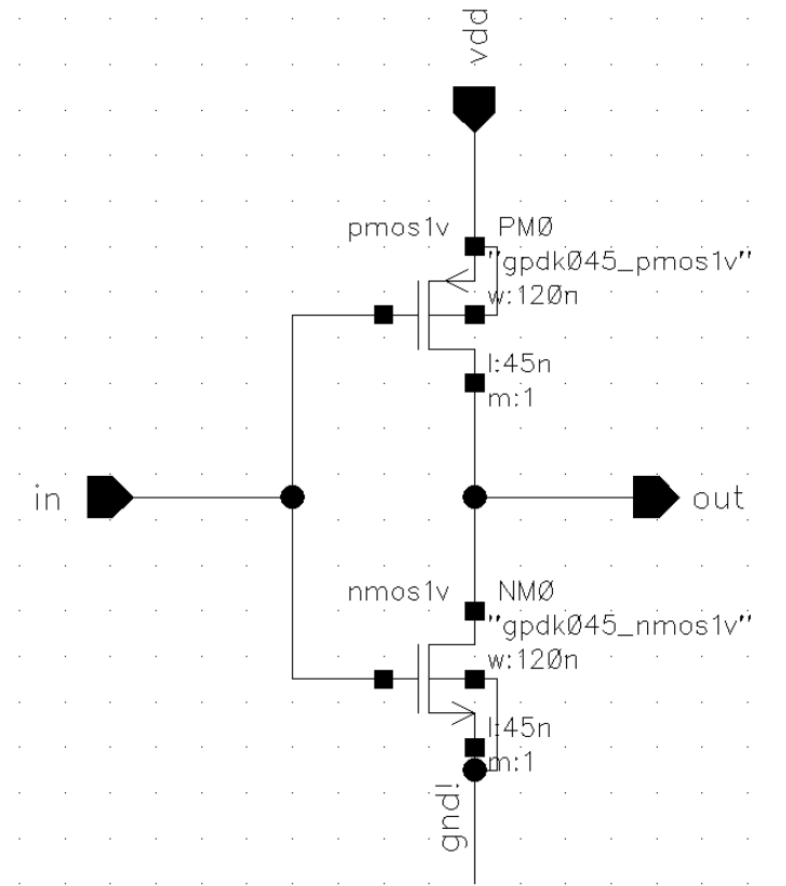
**D-FF Schematic**

# Asynchronous Clock Generator (3/5)



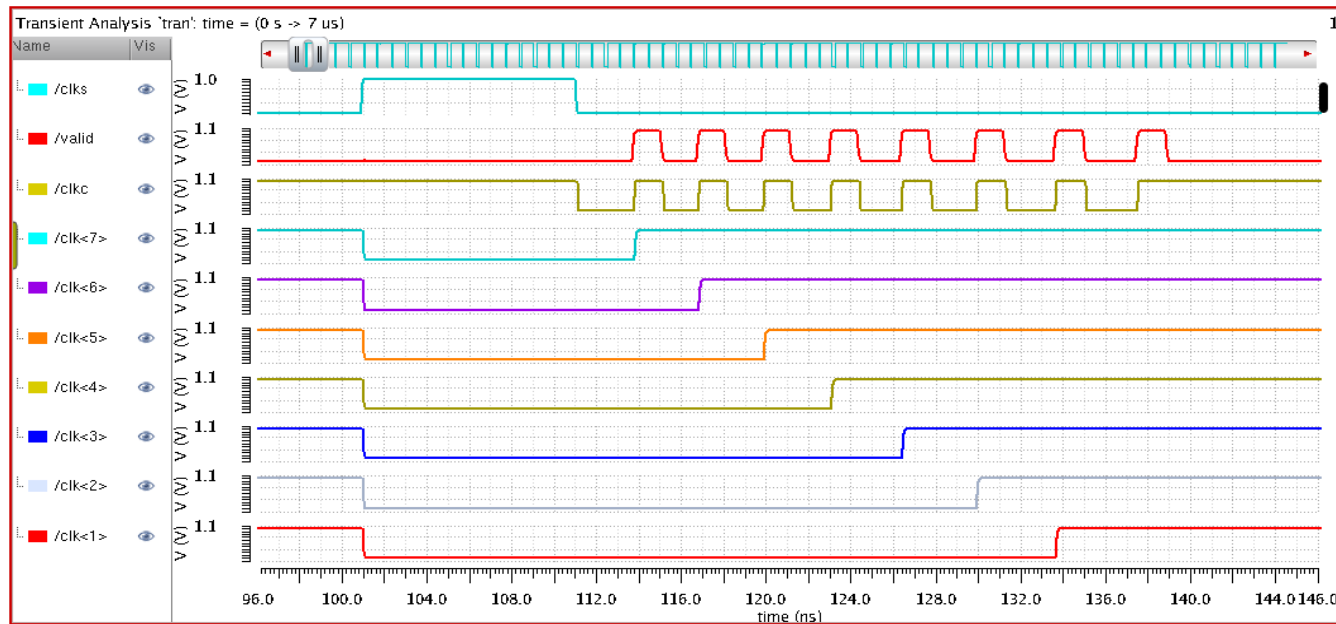
**3x1 OR Gate**

# Asynchronous Clock Generator (4/5)



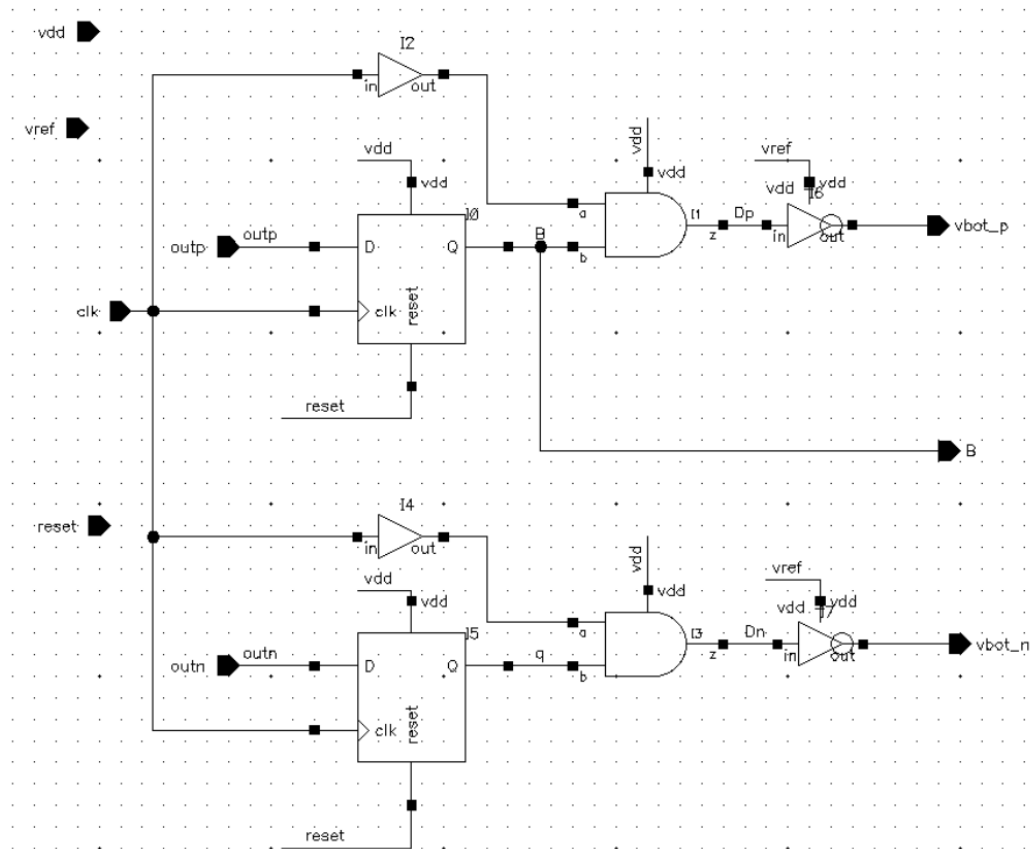
**Inverter Schematic**

# Asynchronous Clock Generator (5/5)



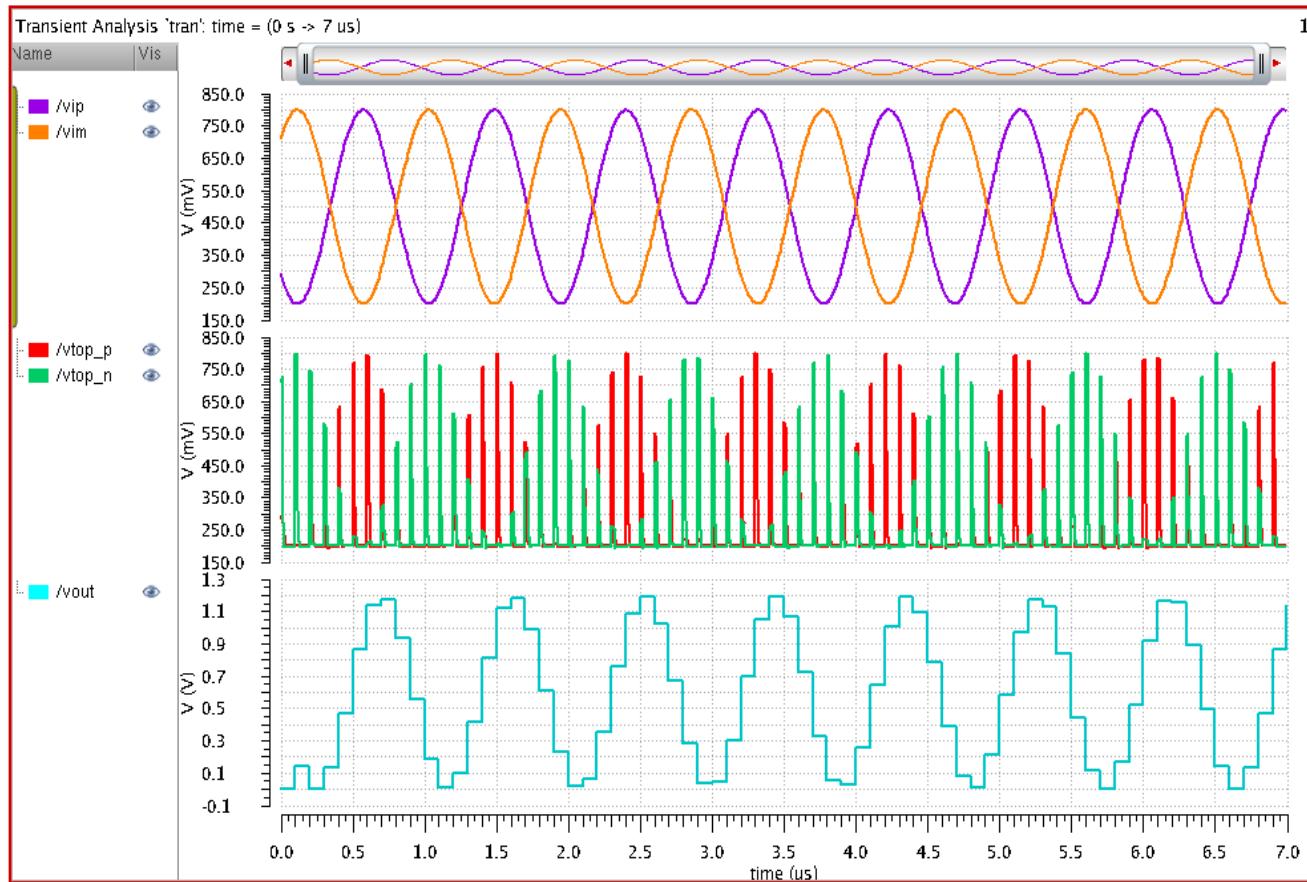
Timing of Clock Generator Output Clocks

# SAR Logic



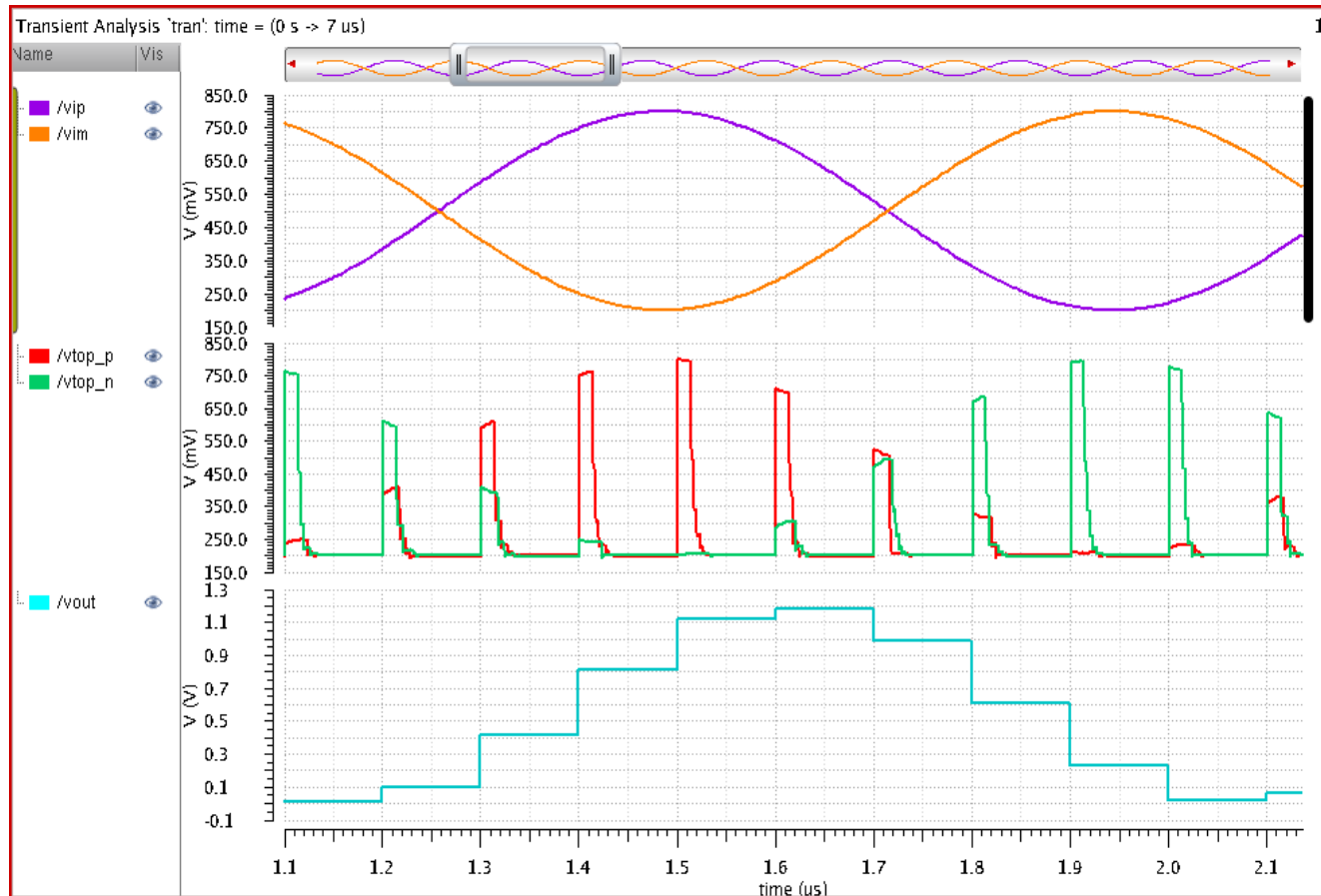
SAR Logic Schematic

# Top-Level Simulation Results (1/5)



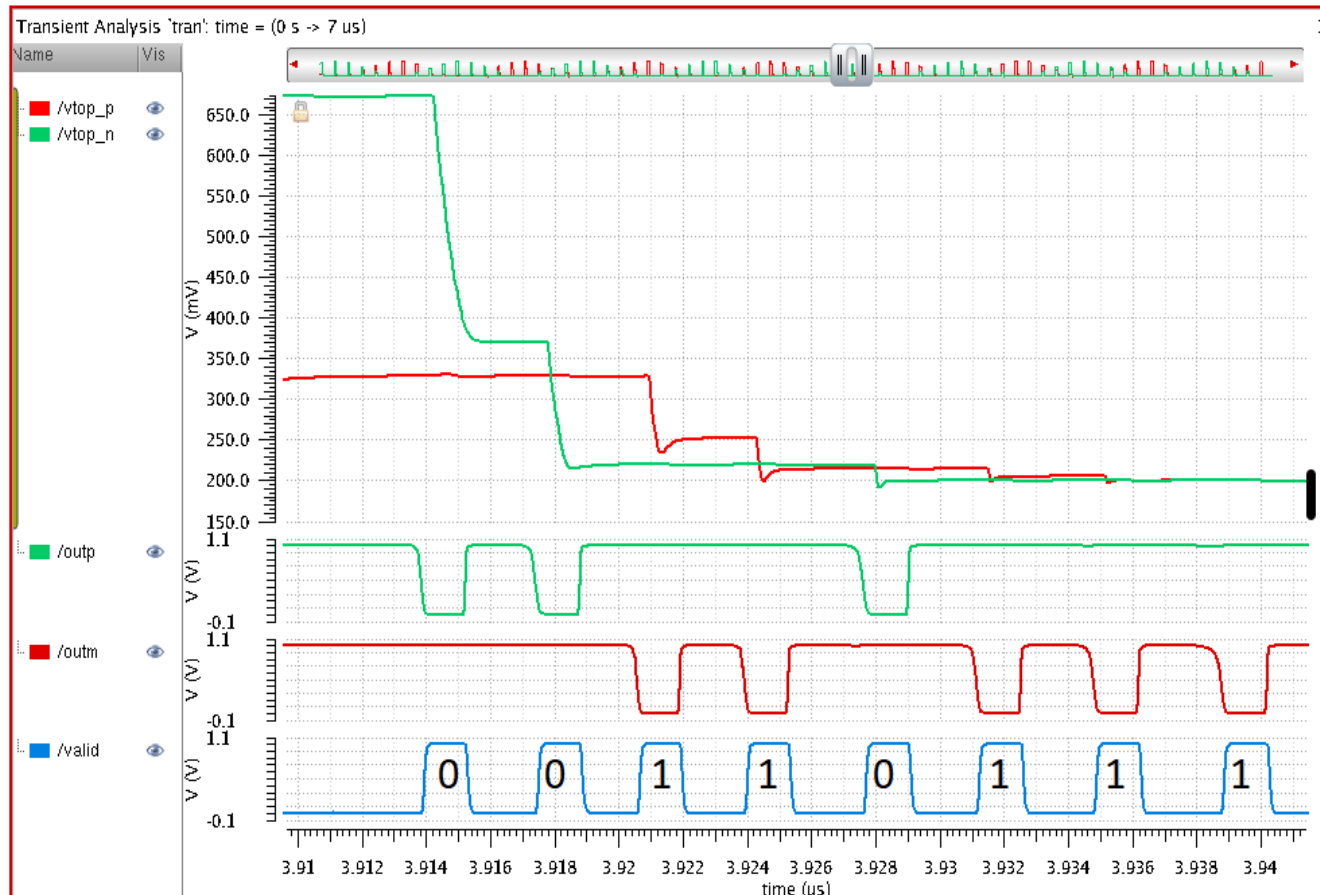
**SAR ADC Transient Simulation Waveforms**

# Top-Level Simulation Results (2/5)



**SAR ADC Transient Simulation Waveforms**

# Top-Level Simulation Results (3/5)



Waveforms of One Complete Conversion

# Top-Level Simulation Results (4/5)

**Spectrum**

Signal/Expr Names

vout

Input wave type: Time Domain Waveform

FFT Input method: Calculate Sample Frequency

Start/Stop Time: 190n 6590n

Sample Count/Freq: 64 10.00M

Window Type: Rectangular

Plot FFT (Units): ☒ dB

Start/End Freq: S 156.3k 5.000M

Signal bins: 0

Peak Sat. Level: 0.0

Harmonics: 1

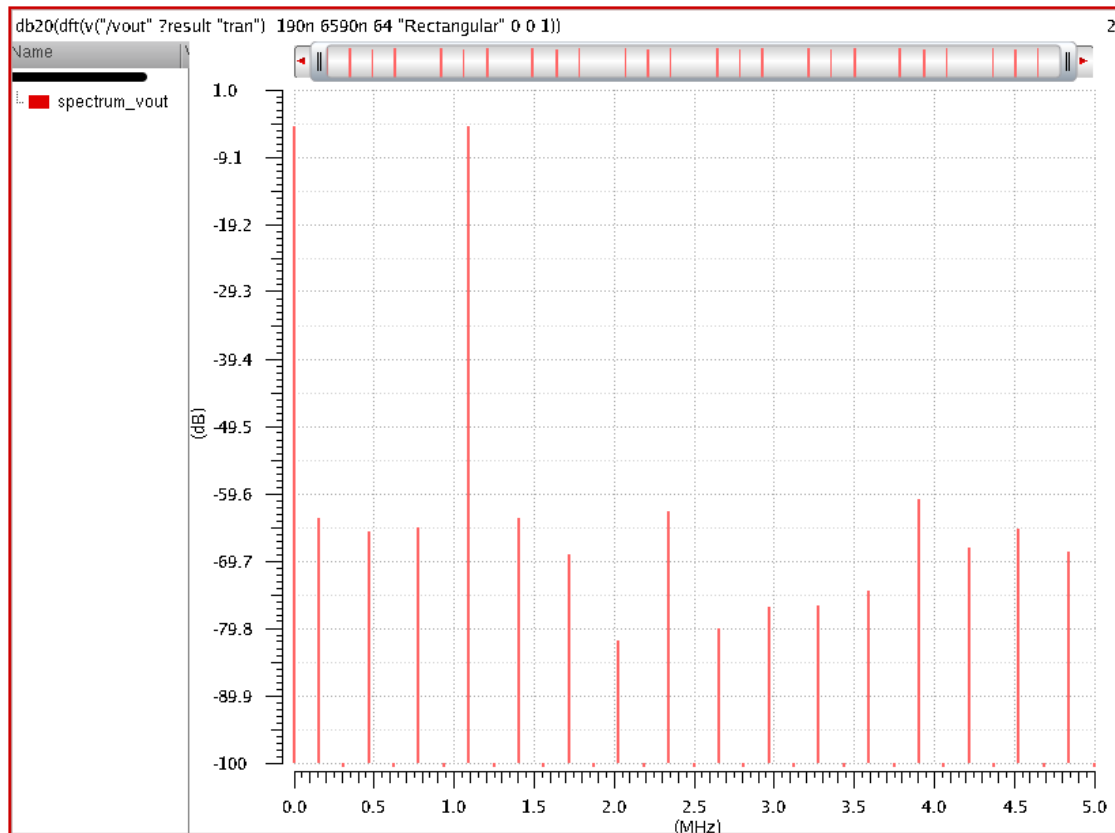
Analysis Type: Signal Analysis

Plot Mode: New Subwindow

Plot

**FFT Setup**

# Top-Level Simulation Results (5/5)



Output Spectrum

## Outputs

| Measurement               | Value            |
|---------------------------|------------------|
| v("/vout" ?result "tran") |                  |
| ENOB                      | 7.9191508 (bits) |
| SINAD                     | 49.436288 (dB)   |
| SNR                       | 49.436288 (dB)   |
| SFDR                      | 55.909182 (dBc)  |
| THD                       | 0 (%)            |
| Signal Power              | -4.5271823 (dB)  |
| DC Power                  | -4.4709707 (dB)  |
| Noise Floor/Bin           | -69.01497 (dB)   |
| Noise Floor/rHz           | -120.81529 (dB)  |

# Summary

| Specification         | Result            |
|-----------------------|-------------------|
| Supply Voltage        | 1V                |
| Input CM Voltage      | 0.5V              |
| Input Range $V_{p-p}$ | 1.2V              |
| Sampling Rate         | 10 MS/s           |
| Sampling Unit Cap $C$ | 2 fF              |
| Attenuation Cap $C_h$ | 65 fF             |
| SINAD                 | 49.44 dB          |
| SFDR                  | 55.9 dBc          |
| ENOB                  | 7.92              |
| Power                 | 10.11 uW          |
| FOM                   | 4.12 fJ/conv-step |

**Table I: Summary of the results**

# Verilog-A Models

# Asynchronous Clock Generator

```
`include "constants.vams"
`include "disciplines.vams"

module prj_DFF(Q, D, clk, reset, vdd);
    output Q;
    input D, clk, reset, vdd;
    electrical Q, D, clk, reset, vdd;
    parameter real vdd_v = 1;
    parameter rst2q_delay = 1p;
    parameter d2q_delay = 0.1n;

    real q_delay = 1p;
    real q_v;

    analog begin

        @(cross(V(clk)-vdd_v/2, +1)) begin
            q_v = V(D); q_delay = d2q_delay;
        end
        if (V(reset)-vdd_v/2 > 0) begin
            q_v = 0; q_delay = rst2q_delay;
        end

        V(Q) <+ transition(q_v, q_delay, 1p, 1p);
    end

endmodule
```

```
`include "constants.vams"
`include "disciplines.vams"

module prj_or2x1(z, a, b, c);
    output z;
    input a, b, c;
    electrical a, b, c, z;

    parameter real vdd = 1.2;
    real z_v;

    analog begin
        if (V(a) > vdd/2 || V(b) > vdd/2 || V(c) > vdd/2) z_v = vdd;
        else z_v = 0;
        V(z) <+ transition(z_v, 1p, 1p, 1p);
    end

endmodule
```

# SAR Logic

```
`include "constants.vams"
`include "disciplines.vams"

module prj_DFF(Q, D, clk, reset, vdd);
    output Q;
    input D, clk, reset, vdd;
    electrical Q, D, clk, reset, vdd;
    parameter real vdd_v = 1;
    parameter rst2q_delay = 1p;
    parameter d2q_delay = 0.1n;

    real q_delay = 1p;
    real q_v;

    analog begin

        @(cross(V(clk)-vdd_v/2,+1)) begin
            q_v = V(D); q_delay = d2q_delay;
        end
        if (V(reset)-vdd_v/2 > 0) begin
            q_v = 0; q_delay = rst2q_delay;
        end

        V(Q) <+ transition(q_v, q_delay, 1p, 1p);
    end

endmodule

`include "constants.vams"
`include "disciplines.vams"

module prj_and2x1(z, a, b);
    input a, b;
    output z;
    electrical a, b, z;

    parameter real vdd = 1;
    real z_v;
    analog begin
        if(V(a)>vdd/2 && V(b) > vdd/2) z_v = vdd;
        else z_v = 0;
        V(z) <+ transition(z_v, 1p, 1p, 1p);
    end

endmodule
```

```
`include "constants.vams"
`include "disciplines.vams"

module prj_inv(in, out, vdd);
    input in, vdd;
    output out;
    electrical in, out, vdd;

    parameter real delta = 1p;
    real v_out;
    real vdd_v;

    analog begin
        vdd_v = V(vdd);
        if(V(in) > vdd_v/2) v_out = 0;
        if(V(in) < vdd_v/2) v_out = vdd_v;
        V(out) <+ transition(v_out, delta, 1p, 1p);
    end

endmodule

`include "constants.vams"
`include "disciplines.vams"

module prj_buf(in, out);
    input in;
    output out;
    electrical in, out;

    parameter real delta = 10n;
    real v_out;
    parameter real vdd = 1;

    analog begin
        if(V(in) > vdd/2) v_out = vdd;
        if(V(in) < vdd/2) v_out = 0;
        V(out) <+ transition(v_out, delta, 1p, 1p);
    end

endmodule
```